

BWStory: Discover & Profile

Test assignment · React Native (Expo) · Android APK

Ashika Mishra · ashikamishra63@gmail.com · +91 81090 55298

Android app (APK)	[Google Drive link to the APK]
Source code	[GitHub repository link]
Screen recording	[Google Drive link to the video]
Portfolio	[Portfolio link]

1. What I built

The brief asked for two screens based on the BWStory app: **Discover** and **Profile**. I kept BWStory's brand colour (deep navy, #143442) and redesigned the screens as a modern short-video news experience. Tapping through the Discover screen leads to two follow-on pages, so the flow feels like a real app:

Screen	What you can do
Discover (the feed)	Scroll a feed of short news videos. The video most in view plays silently as a preview. Like, comment, share or save a story, and tap a circle at the top to open a reporter's latest story.
Full-screen video	Watch with sound, and tap to pause. Like, save or share from the side bar, read the full summary, follow the reporter, write a comment, or jump to the next story.
Reporter profile	See the reporter's photo, bio, follower numbers and all their stories. Follow them, and the count updates straight away.
Your profile	The same layout, showing your own photo, bio, profession and location.
Edit profile (Update Account)	Change your photo, name, gender, profession, location and bio. Mistakes are pointed out under the field, the bio has a live word counter, and the app asks before throwing away unsaved changes.

Search, Create and Stories are placeholder tabs, as the brief covers only Discover and Profile. People, headlines and numbers are sample content; the videos are free public test clips.

2. How to install the app (Android)

1. On your Android phone, open the **APK link** above and download `BWStory.apk` .
2. Open the downloaded file. If Android says installs from this source aren't allowed, tap **Settings** and turn on **Allow from this source** for your browser or Files app, then go back.

3. Tap **Install**. If Google Play Protect shows a warning, tap **More details** → **Install anyway**. This appears because the app isn't on the Play Store yet.
4. Open **BWStory** from your app list. You need an internet connection for the videos.

To remove it later, press and hold the app icon and tap **Uninstall**. The app asks for **no permissions** at install time. Choosing a profile photo uses Android's built-in photo picker, which doesn't need storage access.

3. How to run the source code (for developers)

You need **Node.js 20 or newer** and the free **Expo Go** app on a phone.

```
git clone [GitHub repository link]
cd react-native-
npm install
npx expo start          # scan the QR code with Expo Go
```

To build the APK yourself:

```
npx eas-cli login
npx eas-cli build -p android --profile preview
```

Checks: `npm run typecheck` and `npm run lint`. The v3 design lives on the branch `design/v3-social`. The first version, which copies BWStory's layout closely, is kept at the tag `v1-classic`.

4. How it's built

Technology

- Expo SDK 57, React Native, TypeScript
- Expo Router for screens and links
- expo-video for playback, expo-image for fast cached images
- Data saved on the phone (no server needed for this brief)

How the code is organised

- Every piece is its own small, reusable component
- One shared set of colours, fonts and spacing
- Work done on separate branches, one per feature

Safety and privacy

- The app blocks camera, microphone and storage permissions it doesn't need, and Android cloud backup of app data is off.
- Everything typed into the profile is cleaned (hidden and look-alike characters are removed) and checked for length and allowed characters.
- Saved data is re-checked when the app opens, so damaged data can't crash it.
- Videos load only over secure HTTPS from an approved list of sources, and links that open a story or profile accept only known IDs.

5. How we can improve it

5.1 A personal feed for every user, using AI

The idea: instead of everyone seeing the same list, each person sees the stories they're most likely to care about, while still being shown important news and new topics.

Step 1: learn what each story is about. When a reporter uploads a story, an AI model reads the headline, the summary and the spoken words (turned into text automatically). It adds topic tags (for example "Metro", "Weather", "Startups"), the place, and a short summary. It also produces a *fingerprint* of the story's meaning, a list of numbers called an *embedding*, so stories about similar things can be found even when they use different words.

Step 2: learn what each person likes, from what they do. The app notes simple signals: how much of a video someone watches, whether they skip it quickly, likes, saves, shares, comments, who they follow, the area they're in and the time of day. Watching to the end counts for much more than a quick scroll past. From this, each person gets their own "taste fingerprint" that keeps updating as they use the app.

Step 3: build the feed in three passes.

1. **Collect** a few hundred candidate stories: from people you follow, from your area, trending now, and similar to what you enjoyed.
2. **Score** each one by how well it matches your taste, how fresh it is and how trusted the reporter is.
3. **Mix** the final list, so it isn't ten stories on one topic. Always include important local news, and keep about one in ten stories new to you, so the feed doesn't get stuck in a bubble.

New users: ask them to pick 3–5 topics and their city when they first open the app, and show popular local stories until the app has learned more.

Keeping it fair and private: add a "Why am I seeing this?" note and buttons for *Show more like this* / *Show less*, let people reset their history, never guess sensitive traits (religion, health, politics), and give verified and fact-checked news priority.

Measuring success: test changes on a small group first (an A/B test) and look at whether people watch stories to the end and come back the next day, not just at clicks.

Other helpful AI features: automatic Hindi and English captions and translations, a one-line summary on every video, spotting duplicate or misleading uploads, and suggesting a headline to reporters as they post.

Possible tools: Supabase (PostgreSQL with the pgvector add-on) to store and search story fingerprints, an embedding model and a speech-to-text model for step 1, and a small server that builds each person's feed. Start with simple rules, then move to a trained ranking model as data grows.

5.2 Making the app load and respond faster

Area	What to do, and why it helps
Videos start instantly	Stream videos in small pieces at several quality levels (called <i>HLS</i>), so playback starts at once and adjusts to the network. Load the first couple of seconds of the <i>next</i> video while the current one plays, and serve all videos from a CDN (servers close to the user).

Images appear immediately	Show a tiny blurred preview first and then the sharp image. Serve images in the exact size the screen needs, in modern formats (WebP/AVIF). Caching is already on (expo-image).
Smooth scrolling	Switch the feed list to <i>FlashList</i> , which reuses screen items instead of creating new ones. Load stories in pages of 10 as the user scrolls, and keep at most one or two video players alive at a time (already done).
Fast app start	Show the last loaded feed from the phone straight away, then refresh it quietly in the background. Load rarely used screens only when they're opened, and trim the fonts and code shipped in the app.
Instant reactions	Update likes, follows and saves on screen before the server replies, undoing only if it fails (partly done). Run animations on the phone's graphics thread (Reanimated), and add small haptic taps (already done).
Slow networks	Add a low-data mode (lower video quality, no auto-preview), queue actions made offline and send them when the connection returns, and show placeholder shapes (skeletons) instead of blank space while loading.
Measure, don't guess	Track app start time, time until the first video frame, and dropped frames with a monitoring tool (for example Sentry), and set targets such as "first video plays in under 1 second on 4G".

5.3 Other next steps

- **Real accounts and data:** sign-in, stories, likes and comments stored in Supabase or Firebase, with video storage.
- **Posting stories:** record or upload from the Create tab, with trimming and captions.
- **Comments, notifications and search** as full features.
- **Hindi and other Indian languages** across the app.
- **Automated tests** for components and full user journeys, run on every change, plus crash reporting.
- **An iPhone version** through TestFlight (needs an Apple Developer account).

6. Known limitations

- Sample content and public test videos, with no server yet. Likes, follows and comments are kept only while the app is open; the profile is saved on the phone.
- Tested on an Android phone. An iPhone build was not produced, as the brief asks for an APK.

Thank you for reviewing my submission. Ashika Mishra · ashikamishra63@gmail.com · github.com/malicious28